

■ PHASE 0 — Prerequisite Mindset (Skip if Already Strong)

■ Goal

You should be comfortable writing clean structured code, not just solving DSA problems.

■ Must Know Before LLD

Functions

Classes

Objects

Basic STL / Collections

Clean coding habits

■ Practice (Warmup)

Implement Stack using Class

Implement Queue using Class

Implement Bank Account class

Implement Student Management mini system

■ Resources

■ YouTube:

CodeWithHarry – OOP in C++

Apna College – OOP Complete Playlist

■ PHASE 1 — OOP MASTER LEVEL (VERY IMPORTANT)

■ Many students rush LLD → Big mistake

■ LLD = Applied OOP

■ 1■■■ OOP Core Concepts (Deep Level)

■ Class & Object

Subtopics:

Constructors (Default, Param, Copy)

Destructor

Static variables

Static methods

Object lifecycle

■ Encapsulation

Private / Public / Protected

Getters Setters

Data hiding

Real world mapping

■ Abstraction

Abstract class

Interface

Pure virtual functions

When to use interface vs abstract class

■ Inheritance (Deep)

Types:

Single

Multilevel

Multiple

Hybrid

Hierarchical

Concepts:

Method overriding

Virtual functions

Diamond problem

■ Polymorphism

Compile time (Overloading)

Runtime (Overriding)

Operator overloading (Optional but good)

■ Practice Questions (MANDATORY)

Implement:

■ Library Management System

■ Hotel Booking Basic Model

■ Online Shopping Basic Model

■ Employee Payroll System

■ Best YouTube Playlists

■ Neso Academy – OOP

- Concepts with Code – OOP LLD Intro
- Gaurav Sen – OOP Basics for Design

■ PHASE 2 — SOLID PRINCIPLES (INTERVIEW FAVORITE)

- 2■■■ SOLID Principles (Extreme Depth)
- S — Single Responsibility Principle

One class → One job

Practice:

Split God class into smaller classes

- O — Open Closed Principle

Open for extension

Closed for modification

Practice:

Payment system → Add new payment without editing old code

- L — Liskov Substitution

Child should behave like parent logically

- I — Interface Segregation

Small interfaces > Big interfaces

- D — Dependency Inversion

Depend on abstraction not concrete class

- Practice Design Tasks

- Payment Gateway with multiple payment types
- Notification system (Email + SMS + Push)
- Logging framework

- Resources

- Gaurav Sen – SOLID Principles
- Concepts with Code – SOLID Playlist

■ PHASE 3 — DESIGN PATTERNS (LLD CORE WEAPON)

- 3■■■ Creational Patterns

Singleton

Factory

Abstract Factory

Builder

Practice:

Logger Singleton

Vehicle Factory

Database Connection Singleton

■ 4■■ Structural Patterns

Adapter

Decorator

Facade

Proxy

Practice:

Payment adapter

Coffee decorator pricing

■ 5■■ Behavioral Patterns

Strategy

Observer

State

Command

Practice:

Notification Observer

Payment Strategy

Order State Machine

■ Resources

■ Refactoring Guru (Website + YouTube)

■ Gaurav Sen – Design Patterns

■ Concepts with Code – Design Patterns

■■ PHASE 4 — CORE LLD INTERVIEW PROBLEMS

■ Beginner LLD Problems

Solve FULL class diagram + code

■ Parking Lot

■ Library System

■ Movie Ticket Booking

■ ATM Machine

■ Car Rental System

■ Intermediate LLD Problems

- BookMyShow
- Food Delivery (Swiggy type)
- Ride Sharing (Uber basic)
- Splitwise
- Chess Game

■ Advanced LLD Problems

- Rate Limiter
- Notification System (Scalable)
- Cache System (LRU / LFU class design)
- Messaging Queue Basic Model

■ Resources

- Gaurav Sen – LLD Problems
- Concepts with Code – LLD Interview Series
- Anuj Bhaiya – LLD Playlist

■ PHASE 5 — UML & DESIGN COMMUNICATION

■ Learn

Class Diagram

Sequence Diagram

Use Case Diagram

■ Resources

- Lucidchart UML Tutorials
- Visual Paradigm UML Basics

■ PHASE 6 — DATABASE + API THINKING (LLD+)

Learn Basics

Entity design

Normalization

REST API basics

DTO concept

Practice

Design:

User Service API

Order Service API

Payment Service API

■ PHASE 7 — MOCK INTERVIEW LEVEL

Full Case Study Practice

Do end-to-end:

- Parking Lot → Diagram → Code → Edge cases
- Uber → Modules → Classes → Flow
- Splitwise → Ledger logic → Settlement

■ IDEAL TIMELINE (REALISTIC)

■ Month 1

OOP + Practice Systems

■ Month 2

SOLID + Design Patterns

■ Month 3

LLD Problems + Mock Interviews

■ TOP PLAYLIST COMBO (If You Follow Only Few)

If limited time:

■ MUST WATCH

Concepts with Code – LLD Playlist

Gaurav Sen – LLD + System Design Basics

Refactoring Guru – Design Patterns

■ EXTREME BONUS (If Targeting Top Product Companies)

Learn:

Thread Safety Basics

Immutability

Basic Concurrency

Caching concepts

■ FINAL INTERVIEW READY CHECKLIST

You are LLD ready if you can:

- Convert problem → Classes
- Apply SOLID naturally
- Use 2–3 design patterns naturally
- Write clean extensible code
- Explain tradeoffs